OOP: Assignment 1 - Car Marketplace System

Name: Tehreem Anwar **Roll No: 25K-0667**

1. Constructors Reasoning:

- **Account:**
  - Parameterized Constructor - initializer list necessary for const id.
  - Default Constructor - for preview as Guest before the user officially signs up.
- **Engine:**
  - Parameterized Constructor - initializer list necessary for const serial.
  - Default Constructor - component of Vehicle class, if engine details not provided, program requires a default constructor call.
- **Category:**
  - Parameterized Constructor - initializer list necessary for const id.
  - Default Constructor - component of Vehicle class, if component details not provided, program requires a default constructor call.
- **Vehicle:**
  - Parameterized Constructor - each vehicle should have a brand, model, year, etc at the time of its existence. This is to avoid ghost vehicles.
- **User:**
  - Parameterized Constructor - initializer list necessary for const id. Each user should have a name, id, email, role, etc at time of existence. This avoids ghost users.
- **Spare Part:**
  - Parameterized Constructor - initializer list necessary for const id. Each user should have a name, id, email, role, etc at time of existence. This avoids ghost users.
- **Listing:**
  - Parameterized Constructor - initializer list necessary for const id.
- **Message:**
  - Parameterized Constructor - text should have some content.
- **Admin Panel:**
  - Parameterized Constructor - initializer list for const key. Each admin should have a name, id, access level at time of creation. Maintains integrity and avoids security breaches.
- **Marketplace:**
  - Parameterized Constructor - marketplace should have listing, users, etc. Initializes count.

2. **Static Reasoning:**

Users:

- **Static Data Member (totalusers):** Acts as a global counter for users. Incremented each time a user is created and remains the same for all users.
- **Static Function (getTotalUsers):** Getter for private data member. Static since **totalusers** is also static. Allows the program to access **totalusers** without calling an object.

Vehicle:

- **Static Data Member (vehicleCount):** Acts as a global counter for vehicles. Incremented each time a vehicle is created and remains the same for all vehicles.
- **Static Function (getvehicleCount):** Getter for private data member. Static since **vehicleCount** is also static. Allows the program to access **vehicleCount** without calling an object.

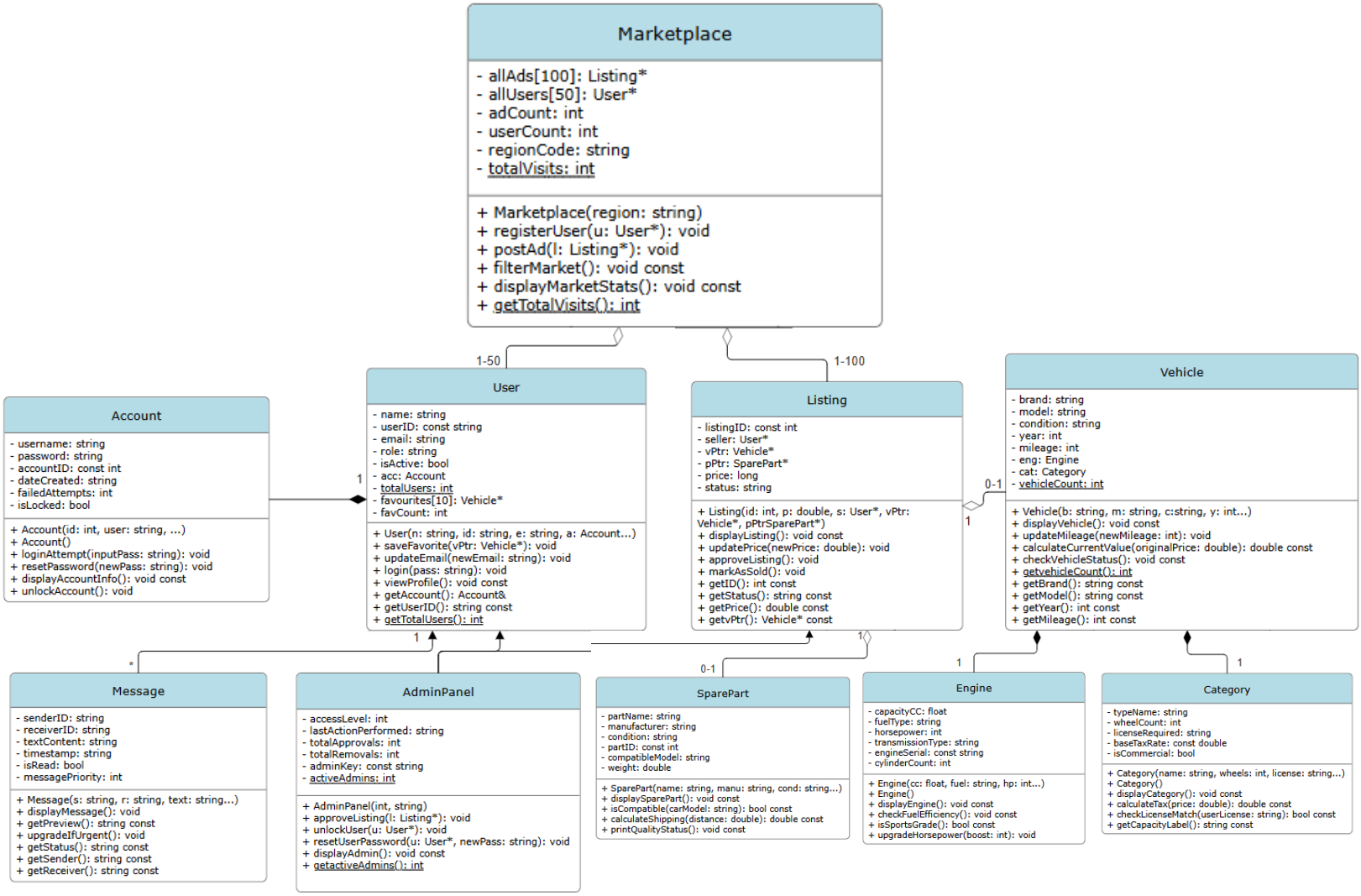
Admin Panel:

- **Static Data Member (activeAdmins):** Acts as a live global counter for admins currently online. Incremented each time an admin logs in and remains the same for all admins.
- **Static Function (getactiveAdmins):** Getter for private data member. Static since **activeAdmins** is also static. Allows the program to access **activeAdmins** without calling an object.

Marketplace:

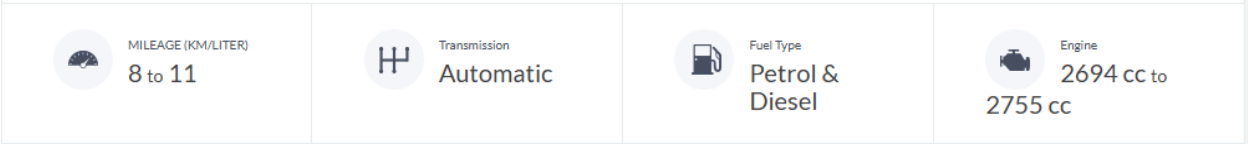
- **Static Data Member (totalVisits):** Acts as a live global counter for current website traffic. Incremented each time a user is registered or views the regional marketplace. While **adCount** and **userCount** are specific to each region (e.g Karachi), **totalVisits** remains the same for all regional markets (e.g Lahore, Islamabad).
- **Static Function (gettotalVisits):** Getter for private data member. Static since **totalVisits** is also static. Allows the program to access **totalVisits** without calling an object. Markets across all regions can view website traffic volume.

3. Class Diagram:

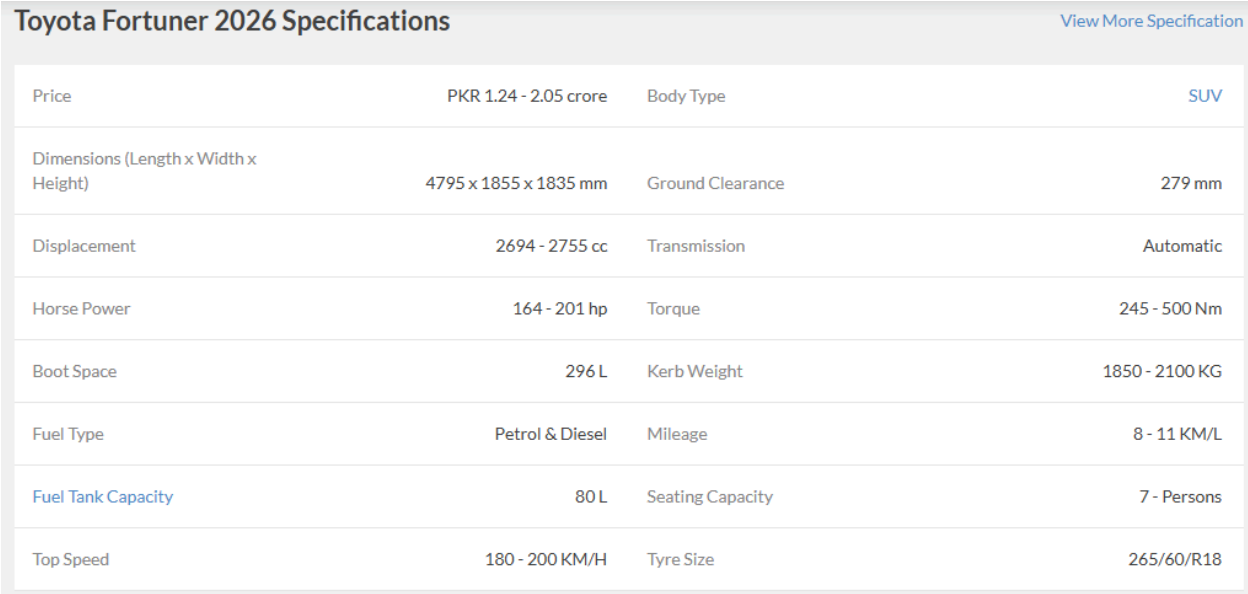


4. Code/Website Snippets:

- Display of Vehicle, Engine, Category details for Car specifications like given in the website.



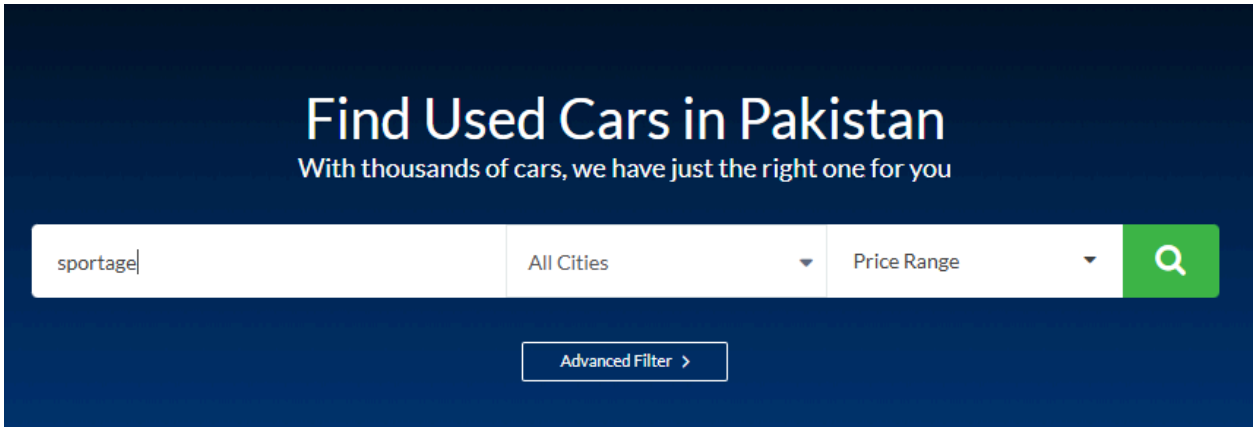

```
// Function 1: Complete Display (Vehicle, Engine, Category)
void displayVehicle() const {
    cout<<"--- "<<year<<" "<<brand<<" "<<model<<" ---"<<endl;
    cout<<"Condition: "<<condition<<" | Mileage: "<<mileage<<"km"<<endl;
    eng.displayEngine();
    cat.displayCategory();
}
```



Toyota Fortuner 2026 Specifications — View More Specification

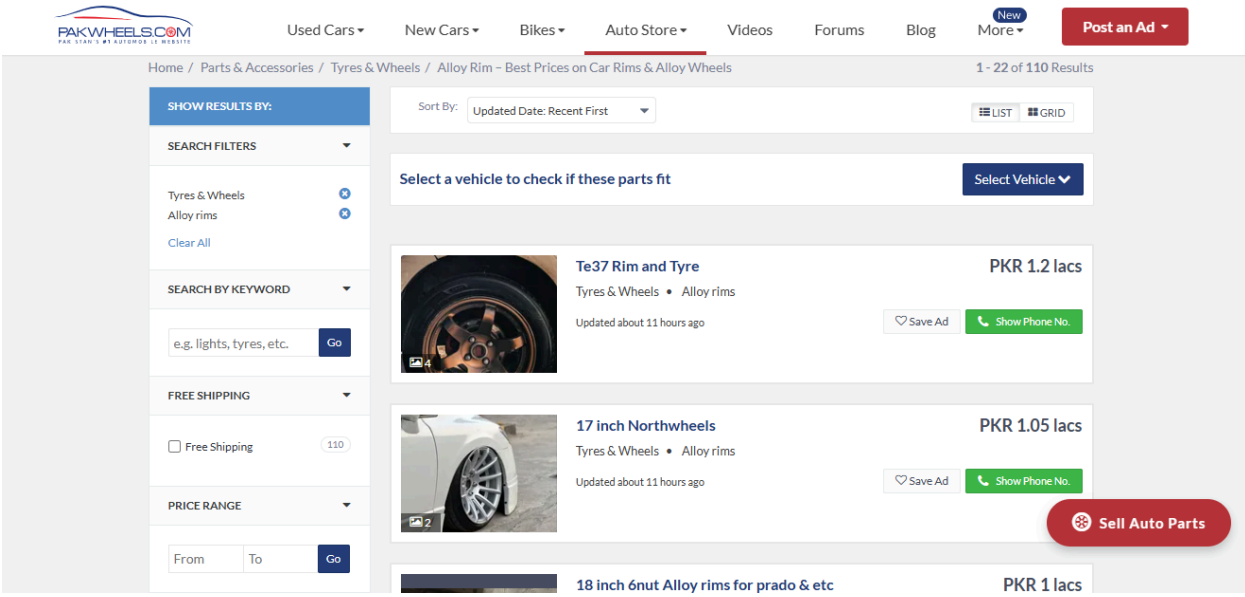
Price	PKR 1.24 - 2.05 crore	Body Type	SUV
Dimensions (Length x Width x Height)	4795 x 1855 x 1835 mm	Ground Clearance	279 mm
Displacement	2694 - 2755 cc	Transmission	Automatic
Horse Power	164 - 201 hp	Torque	245 - 500 Nm
Boot Space	296 L	Kerb Weight	1850 - 2100 KG
Fuel Type	Petrol & Diesel	Mileage	8 - 11 KM/L
Fuel Tank Capacity	80 L	Seating Capacity	7 - Persons
Top Speed	180 - 200 KM/H	Tyre Size	265/60/R18

- Searching and filtering by brand/price/mileage like given in the website.



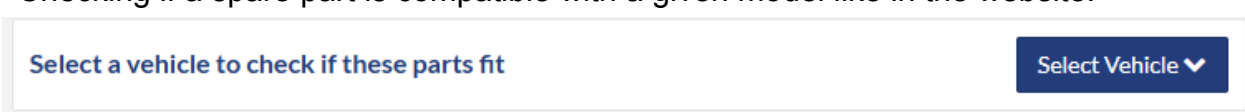
```
// Function 3: Search Logic
void filterMarket() const {
    int choice;
    cout<<"--- Search & Filter Menu ---"<<endl;
    cout<<"1. Search by Brand"<<endl;
    cout<<"2. Search by Year"<<endl;
    cout<<"3. Search by Max Price"<<endl;
    cout<<"4. Search by Max Mileage"<<endl;
    cout<<"Enter choice: ";
    cin>>choice;

    switch(choice) {
        case 1: {
            string b;
            cout<<"Enter Brand: "; cin>>b;
            for(int i=0; i<adCount; i++) {
                if(allAds[i]->getvPtr()!=nullptr && allAds[i]->getvPtr()->getBrand()==b) {
                    allAds[i]->displayListing();
                    cout<<endl;
                }
            }
            break;
        }
        case 2: {
            int y;
            cout<<"Enter Year: "; cin>>y;
            for(int i=0; i<adCount; i++) {
                if(allAds[i]->getvPtr()!=nullptr && allAds[i]->getvPtr()->getYear()==y) {
                    allAds[i]->displayListing();
                    cout<<endl;
                }
            }
            break;
        }
        case 3: {
            double p;
            cout<<"Enter Max Price: "; cin>>p;
            for(int i=0; i<adCount; i++) {
                if(allAds[i]->getPrice()<=p) {
                    allAds[i]->displayListing();
                    cout<<endl;
                }
            }
            break;
        }
        case 4: {
            int m;
            cout<<"Enter Max Mileage: "; cin>>m;
            for(int i=0; i<adCount; i++) {
                if(allAds[i]->getvPtr() != nullptr && allAds[i]->getvPtr()->getMileage()<=m) {
                    allAds[i]->displayListing();
                    cout<<endl;
                }
            }
            break;
        }
        default:
            cout<<"Invalid choice."<<endl;
    }
}
```

- Marketplace can post listings of ads like in the website.



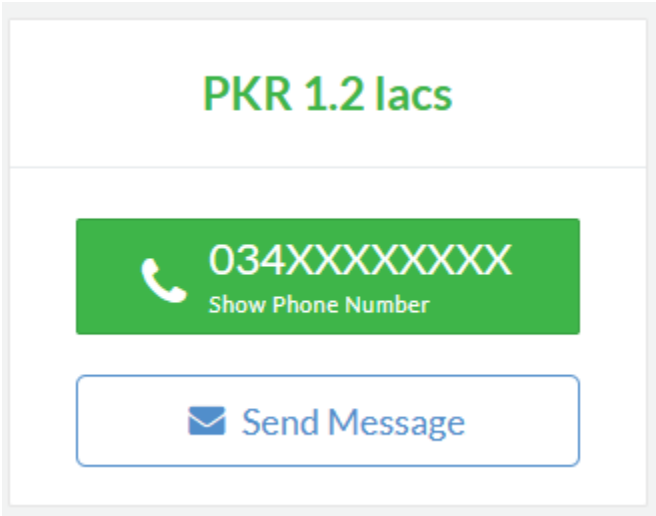

```
// Function 2: Post an Ad (Limit 100 per Region)
void postAd(Listing* l) {
    if(adCount<100){
        allAds[adCount++]=l;
        totalVisits++;
        cout<<"Ad posted successfully in region: "<<regionCode<<endl;
    }else{
        cout<<"Error: Region Ad limit reached."<<endl;
    }
}
```

- Checking if a spare part is compatible with a given model like in the website.




```
// Function 2: Compatibility Check (If it matches OR is a universal part)
bool isCompatible(string carModel) const {
    if(carModel == compatibleModel || compatibleModel == "Universal") {
        return true;
    }
    return false;
}
```

- Message option to chat between Buyer and Seller like given in the website.




```
// 8. Communication
class Message {
    string senderID;
    string receiverID;
    string textContent;
    string timestamp;
    bool isRead;
    int messagePriority;
    public:
        //Parameterized Constructor (text should have some content)
        Message(string s, string r, string text, string time, int priority)
        : senderID(s), receiverID(r), textContent(text), timestamp(time), isRead(false), messagePriority(priority) {}

        // Function 1: Display with Read-Status Update
        void displayMessage() {
            isRead=true;
            cout<<"["<<timestamp<<"] From: "<<senderID<<" | Priority: "<<messagePriority<<endl;
            cout<<"Content: "<<textContent<<endl;
        }

        // Function 2: Preview Generator (Logic for Inbox/Notification View)
        string getPreview() const {
            if(textContent.length()>15){
                return textContent.substr(0,15)+"...";
            }
```